Conversation folding: benchmark results

llm-gateway, OpenRouter, 2026-10-23. Main model openai/gpt-4.1-mini ($0.40 in, $0.10 cached, $1.60 out per million tokens); summarizer openai/gpt-4.1-nano ($0.10 / $0.40).

Finding. Priced the way OpenAI charges, with its prompt-cache discount, folding cut the provider's bill by **51%–54%** in all 4 long runs. Facts that stay fixed were all kept, through 200 calls and the fold cap. But where facts change mid-conversation, the folded tenant lost the latest *owner* right after a fold in both runs, and got it wrong on 75 answers.

Method

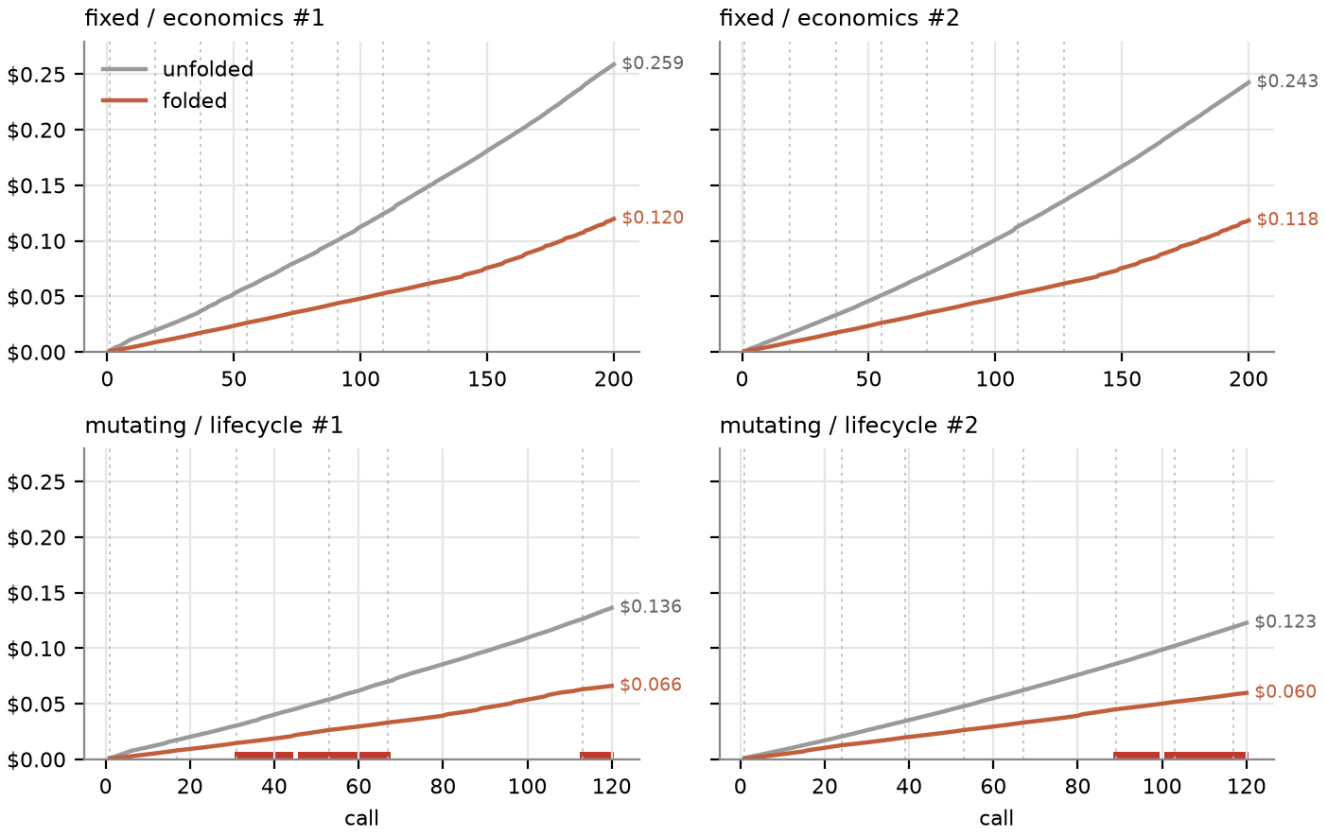
tools/fold_retention.py sends every call twice through the gateway: once to a tenant without folding and once to a tenant with it, with the same model, the same messages and temperature 0. Which tenant goes first alternates each call. The conversation states ten facts, corrects some of them and mixes in a second project to confuse them with (5-fold-engine's scenarios). Answers are scored by strict typed equality. A fact counts towards retention only if it was last stated inside the turns the summary replaced and the unfolded tenant got it right on the same call.

Billed is the gateway's own ledger, which charges every prompt token at the full price. **With caching** is what the provider charges: cached prompt tokens (reported by the provider as usage.prompt_tokens_details.cached_tokens) at the cached rate. Summarizer calls are priced at full price, which if anything overstates the folded tenant. The *economics* profile uses the production fold thresholds; *lifecycle* lowers them to fold often.

Long runs

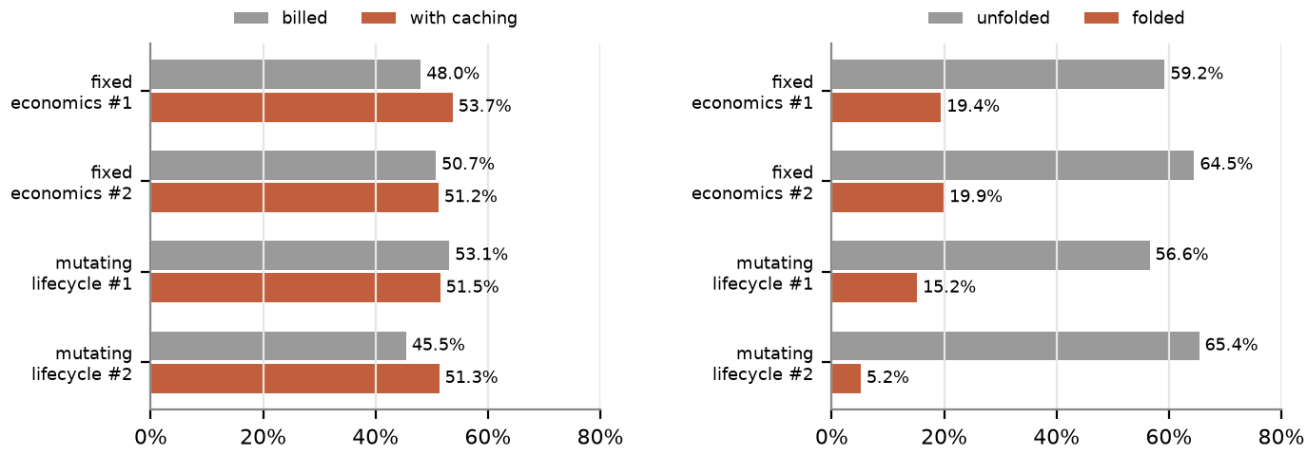
Run	Calls	Reached	Folds	Facts kept	Folded accuracy	Cost unfolded to folded	Saving	Billed saving
fixed / economics #1	200	frozen	8	1492 / 1492	100.0%	$0.259 to $0.120	53.7%	48.0%
fixed / economics #2	200	frozen	8	1492 / 1492	100.0%	$0.243 to $0.118	51.2%	50.7%
mutating / lifecycle #1	120	re-folded	6	772 / 816	96.3%	$0.136 to $0.066	51.5%	53.1%
mutating / lifecycle #2	120	re-folded	8	836 / 867	97.4%	$0.123 to $0.060	51.3%	45.5%

Cumulative provider cost, per run



With caching. Dashed lines mark committed folds; red marks under the axis are calls where the folded tenant got a counted fact wrong.

Saving, and share of prompt tokens served from cache



Left: billed saving vs the saving with caching. Right: an unfolded conversation only grows at its end, so most of it is cached; a fold rewrites its start, so little is. The unfolded prompt is still several times larger, so caching narrows the saving less than one might expect.

Facts the folded tenant lost

Run	Field	Latest value	First wrong call	Calls wrong
mutating / lifecycle #1	owner	Noor	31	44
mutating / lifecycle #2	owner	Ines	89	31

Only the owner field was lost, both times right after a fold, and never recovered. Pinned values did not prevent it. Run 1's correction ("Noor replaces Mira as owner") matches no pin pattern, so the name was never pinned. Run 2's ("owner is now Ines", sent at the fold cap) does match one, so the summary had to keep "Ines" somewhere, yet the answers

were still wrong: a pin keeps a value, not which field it belongs to. The result files do not record the wrong answers themselves. Both mutating runs exit 1, the harness's regression code.

Earlier short runs (50 calls, same models)

Run	Facts kept	Accuracy, unfolded / folded	Billed, unfolded to folded	Saving	Folds committed
fixed, lifecycle	356 / 356	100% / 100%	$0.136 to $0.088	36%	4 (46 unprofitable)
mutating, lifecycle	322 / 322	100% / 100%	$0.128 to $0.083	35%	4 (3 rejected, 43 unprofitable)
fixed, economics	356 / 356	100% / 100%	$0.127 to $0.076	41%	3 (3 unprofitable)

None of these reached the fold cap, and their savings are billed (no cache discount).

Limits

• The scenario is synthetic: ten facts and one distractor project. It shows what a cheap summarizer did here, not that summaries keep facts in general.
• Each long run was done twice. The repeats agree on the shape (saving 51–54%, the owner lost after a fold) but not the details: which fold drops it and when differ, even at temperature 0.
• The economics profile froze at the fold cap and never re-folded within 200 calls.
• One provider and one model pair. Groq's run (gpt-oss-120b) is in the repository README; its other runs hit the account's daily token limit.

Result files: benchmark_results/fold_retention_openrouter_*_20261023T1*.json in llm-gateway.